8. Conditional Multivariate Gaussian, In Depth

Let's focus on conditional multivariate gaussian distributions. First, drop the `conditional` part and just focus on the multivariate gaussian distribution. Actually, drop the `multivariate` part and just focus on the gaussian.

8.1. Gaussian

The gaussian is typically represented compactly as follows.

$$X \sim \mathcal{N}(\mu, \sigma^2)$$

where

- X is a single random variable
- μ is the mean of X
- σ^2 is the variance of X

The statement, $X \sim \mathcal{N}(\mu, \sigma^2)$, says that X comes from a gaussian distribution with a mean μ and variance σ^2; μ and σ^2 are called the `parameters`. Once you know the parameters of a gaussian, you can calculate the probability of any value through the probability density function `PDF`. The PDF of a gaussian is defined as follows.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

8.2. Multivariate gaussian

The multivariate gaussian is represented as follows.

$$\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

- $\mathbf{X}$ is now a vector of random variables
- $\boldsymbol{\mu}$ is a vector of means
- $\boldsymbol{\Sigma}$ is a covariance matrix

The PDF of a multivariate gaussian is as follows.

$$f_{\mathbf{X}}(x_1, \ldots, x_k) = \frac{\exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^{\mathrm{T}}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)}{\sqrt{(2\pi)^k|\boldsymbol{\Sigma}|}}$$

8.3. Motivation

Ok, let's stop here. We know the gaussian and we know the multivariate gaussian. Additionally, it's awesome that if we know the parameters of the gaussian, then we have a way to estimate the probability of any value. What's the point?

The parameters and PDF can be used to compute how likely it is that the data came from the model, where `the model` refers to the gaussian distribution (univariate or multivariate form). Just think of the univariate case where we have two models, $\mathcal{N}_1(0, 1)$ and $\mathcal{N}_2(10, 1)$. If we observe a bunch of values close to zero (e.g. 0.1, -0.1, 0.001, -0.03), which model $\mathcal{N}_1$ or $\mathcal{N}_2$ do you think best explains the data? Obviously $\mathcal{N}_1$ since its mean is closer to the average of what we are seeing of the observations.

We can `score` the models $\mathcal{N}_1$ and $\mathcal{N}_2$ based on the data and the PDF. Note that the models are the same but `parameterized` differently. It is said that $\mathcal{N}_1$ and $\mathcal{N}_2$ come from the `same family of distribution`, here, gaussian. The goal of judging whether $\mathcal{N}_1$ or $\mathcal{N}_2$ best explains the data reduces to finding the parameters that maximizes the probability of the data given the model and parameters. This operation is called `fitting a model to the data`. In more precise methodological terms, it can take on the form of maximum likelihood estimation `MLE` or maximum a posteriori `MAP` estimation.

Still, what does any of this have to do with conditional multivariate gaussian distributions? Imagine that you have a multivariate gaussian data set $\mathbf{X} = \{X_1, X_2, X_3\}$, and you have a hunch that it is likely X_2 is dependent on $X1$ or X_3. Let's denote $\mathcal{N}_{X_2|X_1}$ to represent the model where X_2 is dependent on X_1 and $\mathcal{N}_{X_2|X_1}$ to represent the model where X_2 is dependent on X_3. We can even have a third model $\mathcal{N}_{X_2|X_1,X_3}$ to say that X_2 is dependent on X_1 and X_3. These models are conditional multivariate gaussian models, and we know that we have a PDF that we can use with parameters to score how well the data will fit to these models. Which of these models most likely explain the data? The conditional multivariate gives a way to judge and score the proposed models.

8.4. Conditional multivariate gaussian

Denote the following.

- $\mathbf{x}$ as a vector of observed values
- $\boldsymbol{\mu}$ as a vector of means
- $\boldsymbol{\Sigma}$ as a covariance matrix

Here's the tricky notation part. Let's say a subset of variables in $\mathbf{X}$ is assumed to be dependent on another mutually exclusive subset of variables. Denote the indices of the former as 1 and the indices of the latter as 2. For example, if

- there are 3 variables X_1, X_2, X_3,
- they are indexed 0, 1, 2, and
- we think X_2 is dependent on X_1 and X_3,

then $1 = [1]$ and $2 = [0, 2]$. With this understanding and notation, we can formulate the conditional multivariate gaussian with partitioning the data and parameters as follows.

- $\mathbf{x} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{bmatrix}$
- $\boldsymbol{\mu} = \begin{bmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{bmatrix}$
- $\boldsymbol{\Sigma} = \begin{bmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{bmatrix}$

Then, $\mathbf{x}_1 | \mathbf{x}_2 = a \sim \mathcal{N}(\bar{\boldsymbol{\mu}}, \overline{\boldsymbol{\Sigma}})$, where

- $\bar{\boldsymbol{\mu}} = \boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\left(\mathbf{a} - \boldsymbol{\mu}_2\right)$
- $\overline{\boldsymbol{\Sigma}} = \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}$
- $\mathbf{x}_1$ corresponds to the variables/indices in 1
- $\mathbf{x}_2$ corresponds to the varialbes/indices in 2

$\mathcal{N}(\bar{\boldsymbol{\mu}}, \overline{\boldsymbol{\Sigma}})$ is just the gaussian parameterized slightly different.

Lastly, it is interesting that $\boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}$ gives the `regression coefficients`!

8.5. Simulation

Let's simulate some data and tests some models. The data is simulated as follows.

- $X_1 \sim \mathcal{N}(1, 1)$
- $X_2 \sim \mathcal{N}(1 + 3.5 \times X_1, 1)$
- $X_3 \sim \mathcal{N}(2, 1)$
- $X_4 \sim \mathcal{N}(3.8 - 2.5 \times X_3, 1)$

Focus on X_2, which only depends on X_1; X_3 and X_4 are simulated only to add `noise` and observe how model fitting will do.

[1]:
```
import numpy as np

np.random.seed(37)

N = 1000

x1 = np.random.normal(1, 1, N)
x2 = np.random.normal(1 + 3.5 * x1, 1, N)
x3 = np.random.normal(2, 1, N)
x4 = np.random.normal(3.8 - 2.5 * x3, 1, N)

data = np.vstack([x1, x2, x3, x4]).T
means = data.mean(axis=0)
mins = data.min(axis=0)
maxs = data.max(axis=0)
cov = np.cov(data.T)
std = np.sqrt(np.diag(cov))
cor = np.corrcoef(data.T)

print('means')
print(means)
print('')
print('mins')
print(mins)
print('')
print('maxs')
print(maxs)
print('')
print('cov')
print(cov)
print('')
print('stddev')
print(std)
print('')
print('correlation matrix')
print(cor)
```

```
means
[ 1.01277839  4.52863965  1.98990172 -1.17391554]

mins
[-2.31079823 -8.33142158 -1.80414214 -8.74552729]

maxs
[ 3.92919388 17.57679341  4.82528779  9.27678294]

cov
[[ 9.63461496e-01  3.36840170e+00 -1.12846545e-02 -5.12464592e-02]
 [ 3.36840170e+00  1.27550651e+01 -9.26050108e-02 -8.56265759e-02]
 [-1.12846545e-02 -9.26050108e-02  9.70507183e-01 -2.46328945e+00]
 [-5.12464592e-02 -8.56265759e-02 -2.46328945e+00  7.25484316e+00]]

stddev
[0.98156075 3.5714234  0.98514323 2.69348161]

correlation matrix
[[ 1.          0.9608716  -0.01167002 -0.01938352]
 [ 0.9608716   1.         -0.02632048 -0.0089013 ]
 [-0.01167002 -0.02632048  1.         -0.9283293 ]
 [-0.01938352 -0.0089013  -0.9283293   1.        ]]
```

8.6. Evaluation

Let's now evaluate the following models.

- $\mathcal{N}_{X_2|X_1}$
- $\mathcal{N}_{X_1|X_2}$
- $\mathcal{N}_{X_2|X_3}$
- $\mathcal{N}_{X_2|X_4}$
- $\mathcal{N}_{X_2|X_1,X_3}$
- $\mathcal{N}_{X_2|X_1,X_4}$
- $\mathcal{N}_{X_2|X_3,X_4}$
- $\mathcal{N}_{X_2|X_1,X_3,X_4}$

[2]:
```
from scipy.stats import norm

def get_index_2(index_1, N):
    return [i for i in range(N) if i not in index_1]

def partition_means(index_1, means, index_2=None):
    index_2 = get_index_2(index_1, len(means)) if index_2 is None else index_2
    m_1, m_2 = means[index_1], means[index_2]
    return m_1, m_2

def partition_cov(index_1, cov, index_2=None):
    index_2 = get_index_2(index_1, cov.shape[1]) if index_2 is None else index_2
    s_11 = cov[index_1][:, index_1]
    s_12 = cov[index_1][:, index_2]
    s_21 = cov[index_2][:, index_1]
    s_22 = cov[index_2][:, index_2]

    return s_11, s_12, s_21, np.linalg.inv(s_22)

def partition_x(index_1, x, index_2=None):
    index_2 = get_index_2(index_1, len(x)) if index_2 is None else index_2
    x_1 = x[index_1]
    x_2 = x[index_2]
    return x_1, x_2

def get_log_proba(index_1, data, means, cov, index_2=None, zero=0.000001):
    m_1, m_2 = partition_means(index_1, means, index_2)
    s_11, s_12, s_21, s_22 = partition_cov(index_1, cov, index_2)
    s = (s_11 - s_12.dot(s_22).dot(s_21))[0, 0]

    log_proba = []
    for x in data:
        x_1, x_2 = partition_x(index_1, x, index_2)
        m = (m_1 + s_12.dot(s_22).dot((x_2 - m_2).T))[0]
        p = norm.pdf(x_1, loc=m, scale=s)
        log_p = np.log(p) if p >= zero else 0.0
        log_proba.append(log_p)

    return sum(log_proba)[0], s_12.dot(s_22)[0]
```

$\mathcal{N}_{X_2|X_1}$

[3]:
```
get_log_proba([1], data, means, cov, [0])
```

[3]: (-1407.7504048724838, array([3.49614563]))

$\mathcal{N}_{X_1|X_2}$

[4]:
```
get_log_proba([0], data, means, cov, [1])
```

[4]: (-1844.815851724664, array([0.26408346]))

$\mathcal{N}_{X_2|X_3}$

[5]:
```
get_log_proba([1], data, means, cov, [2])
```

[5]: (-3503.3620421459736, array([-0.09541919]))

$\mathcal{N}_{X_2|X_4}$

[6]:
```
get_log_proba([1], data, means, cov, [3])
```

[6]: (-3503.951768440483, array([-0.01180268]))

$\mathcal{N}_{X_2|X_1,X_3}$

[7]:
```
get_log_proba([1], data, means, cov, [0, 2])
```

[7]: (-1406.293970788458, array([3.49550407, -0.05477492]))

$\mathcal{N}_{X_2|X_1,X_4}$

[8]:
```
get_log_proba([1], data, means, cov, [0, 3])
```

[8]: (-1407.1468441245966, array([3.49683168, 0.0128981]))

$\mathcal{N}_{X_2|X_3,X_4}$

[9]:
```
get_log_proba([1], data, means, cov, [2, 3])
```

[9]: (-3495.601209966525, array([-0.90717683, -0.319823]))

$\mathcal{N}_{X_2|X_1,X_3,X_4}$

[10]:
```
get_log_proba([1], data, means, cov, [0, 2, 3])
```

[10]: (-1405.4323728994732, array([3.49205536, -0.16036643, -0.04158602]))

8.7. Summary

Some interesting observations.

- $\mathcal{N}_{X_2|X_1}$ is a better fit than $\mathcal{N}_{X_1|X_2}$ as expected
- Any time we have X_1 with other variables in the conditioning set, the scores goes up (overfitting is at play here; we could counter overfitting if we can find a way to regularize)
- Without X_1 in the conditioning set, the scores goes down
- Even though having X_3 and X_4 raises the score slightly, their regression coefficients are nearly zero when X_1 is also in the conditioning set